Web Application Assessment Report

=

Company Logo Here

=

Prepared For:
NepaShop Pvt. Ltd.

Prepared By:
Baliram Sah
Penetration Tester Intern

Target:
SmallMart (shop.rezydev.com)

Report Version:
"1.0"

Report Date:
August, 2026

Report Classification:
Confidential

Document of Control

Table 1: Assessment Information

Document Information	Details
Document Type	WallMart Assessment Report
Target Host	Shop.rezydev.com
Report Version	1.0
Classification	Confidential
Prepared By	Baliram Sah
Reviewed By	---
Approved By	---
Assignment Date	August, 2026
Report Date	August, 2026

Confidentiality Statement

This report contains confidential and sensitive information regarding penetration testing and vulnerability assessment, conducted on the SmallMart. The information contained in this document is intended solely for educational assessment and authorized security testing purposes. This report must not be distributed, reproduced, published or disclosed to any unauthorized personnel without the prior consent of authorized authority.

Disclaimer

This assessment was conducted solely for educational and authorized laboratory purpose as a part of the network security training assignment. Testing was limited to the approved targets: *.rezydev.com excluding rezydev.com. No unauthorized systems, networks or third party infrastructure were intentionally targeted. The findings reflect the security posture of the systems at the time of testing and may change due to updates and configuration changes. The tester is not responsible for any unauthorized use of the techniques or information contained in this report. The assessment does not guarantee that all the vulnerabilities or weakness have been identified during security assessments.

Table of Contents

Document of Control....2
Confidentiality Statement....2
Disclaimer....2
1. Introduction....4
2. Assessment Scopes....4
  2.1. In-Scope Assets....4
  2.2. Out of Scope....4
3. Objectives....5
4. Methodology....5
  4.1. Planning and Scoping:....5
  4.2. Information Gathering:....5
  4.3. Vulnerability Analysis (Scanning and Enumeration)....6
  4.4. Exploitation and Post Exploitation....6
  4.5. Risk Severity Rating Assessment....6
  4.6. Reporting....7
5. Executive Summary....7
  5.1. Overview....7
  5.2. Key Findings Summary....7
  5.3. Critical Findings....8
6. Findings Details....9
  F-01: Server-Side Command Injection in administrative terminal....9
  F-02: SQL Injection in product ID endpoints leading to database dump....11
  F-03: Insecure Password Storage in unsalted MD5 hashing....13
  F-05: Sensitive Data exposure in source code comment left by developer....16
  Sensitive Data in HTTP Response Header....18
7. Conclusion....20

1. Introduction

This report presents the findings of a web application penetration test conducted against the SmallMart platform operated by NepaShop Pvt. Ltd. The assessment was performed in accordance with the organization's CTF guidelines, targeting all assets within the *.rezydev.com scope. The primary objective was to identify, validate, and document security vulnerabilities affecting the confidentiality, integrity, and availability of the SmallMart application and its underlying infrastructure. All testing was conducted responsibly with the X-Research-Header: baliram, attribution header included in all requests. Findings are presented with full technical detail, CVSS v4.1 scoring, proof-of-concept evidence, and actionable remediation guidance.

2. Assessment Scopes

2.1. In-Scope Assets

The following assets are authorized for security testing:

Assets	Type	Notes
*.rezydev.com	Web Application	All subdomains, Primary: shop.rezydev.com (SmallMart Nepal)

2.2. Out of Scope

The following are strictly out of scope and must not be tested:

- rezydev.com (main domain)
- Any third-party services or infrastructure not owned by NepaShop Pvt. Ltd.
- Any physical or social engineering attacks.
- Denial-of-Service (DoS/DdoS) testing.
- Automated high-volume scanning that impacts service stability.

3. Objectives

The primary objectives are :

- Identify all exploitable vulnerabilities within *.rezydev.com accessible to an external unauthenticated or low-privilege attacker

- Assess the effectiveness of existing security controls: authentication, input validation, session management, and access controls
- Evaluate the depth of compromise achievable by chaining identified vulnerabilities
- Provide prioritized, actionable remediation recommendations with CVSS v4.1 severity ratings
- Document all findings with sufficient detail to enable accurate reproduction and verification of fixes

4. Methodology

4.1. Planning and Scoping:

The initial phase involved defining the scope and objectives of the assessment, identifying target application and preparing the testing environment. The assessed boundaries were established to ensure the testing activities performed within the authorized scope.

4.2. Information Gathering:

The information gathering phase focused on understanding the application's architecture, technologies, and exposed attack surfaces. Activities performed:

- ► Technology fingerprinting
- ► Identifications of application endpoints
- ► Analysis of JavaScript resources.
- ► Review of publicly accessible resources.
- ► Directory and file enumeration.

4.3. Vulnerability Analysis (Scanning and Enumeration)

The identified application components were systematically tested for known security weakness using both automated and manual techniques. Each finding was verified to eliminate false positives.

Testing Included:

- ► Authentication Testing

- Authorization Testing
- Session management Assessment
- Input validation testing
- REST API security testing
- Client-Side security testing
- Security configuration review.

4.4. Exploitation and Post Exploitation

The identified vulnerabilities were exploited in a controlled environment to determine their practical impact. Exploitation was performed to demonstrate the existence of vulnerabilities without causing unnecessary disruption to the application. After exploitation post-exploitation is done to further identify the impacts.

Testing included verification of the vulnerabilities such as:

- SQL Injection
- OS Command Injection
- Security Misconfiguration
- Information Disclosure via source code

4.5. Risk Severity Rating Assessment

All identified vulnerabilities were assessed using CVSS v4.1 based on their exploitability, impact on the confidentiality, integrity, and availability (CIA). Each finding was assigned a severity level: Critical, High, Medium, Low and Information to prioritize remediation according to its potential security and business impact.

Severity	CVSS Score	Description
Critical	9.0-10.0	Immediate remediation required,. May lead to complete compromise of the application
High	7.0-8.9	Significant security risk requiring prompt remediation
Medium	4.0-6.9	Moderate security risk that should be addressed in a timely manner
Low	0.1-3.9	Limited security impact but should be corrected where practical
Informational	0	Best practice recommendations without direct security impact

4.6. Reporting

The final phase involved documenting all confirmed vulnerabilities, assessing their severity, and providing actionable remediation recommendations.

Each vulnerabilities includes:

- Vulnerability Title, CVSS Score, Severity
- Description
- Affected Components
- Impact, Reference
- Proof of Concepts (PoC)
- Remediation recommendations

5. Executive Summary

5.1. Overview

This report cotains the document of the service enumeration and vulnerability assessments performed against SmallMart (shop.rezydev.com) operated by NepaShop Pvt. Ltd. The assessment was performed in an authorized Capture The Flag (CTF) environment designed to simulate real-world attack scenarios. Testing was conducted in accordance with industry standard methodologies including OWASP Testing Guide.

5.2. Key Findings Summary

The summary of key identified vulnerabilities are shown in below table:

Vuln. ID	Findings	Severity	CVSS Score
F-01	Server-Side Command Injection in administrative Terminal	Critical	9.3
F-02	SQL Injection in product ID endpoints leading to database dump	High	8.8
F-03	Insecure Password Storage in unsalted MD5 hashing	High	7.1
F-04	API endpoints exposed in JavaScript	Medium	5.3
F-05	Sensitive Data exposure in source code comment left by developer	Medium	5.3

F-06	Sensitive Data in HTTP Response Header	Medium	5.3

The assessment identified a total of eight security vulnerabilities across the application stack, ranging from informational disclosures to critical OS command execution vectors. The following table summarizes the severity distribution:

Severity Level	**Count**
Critical	1
High	2
Medium	3
Low	0
Information	0

5.3. Critical Findings

Two vulnerabilities were identified as Critical severity, enabling a complete of the application and underlying server infrastructure:

1. **SQL Injection:** The product detail endpoint allowed authenticated users to extract the entire application database through boolean-based SQL injection. This vulnerability exposed sensitive information including user credential, administrative accounts and application flags.
2. **OS Command Injection:** The administration log viewer functionality permitted authenticated administrators to execute arbitrary operating system commands on the server. This flaw enabled server compromise, including read access to sensitive system files.

6. Findings Details

F-01: Server-Side Command Injection in administrative terminal

Server-side Command Injection in Administrative Terminal	
CVSS Score	9.3 (Critical)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:H/ SC:N/SI:N/SA:N

Affected Component	Administrative Terminal View-logs filter panel
Affected Host	https://shop.rezydev.com/view-logs

Description:
The admin panel's "Filter logs" feature passess user-supplied filters directly to a shell grep command without sanitization. By injecting semicolons and arbitrary shell commands, an attacker with admin access achieves full Remote code Execution (RCE) as the web_svc user. The server file system was fully enumerated and sensitive flag files were read directly.

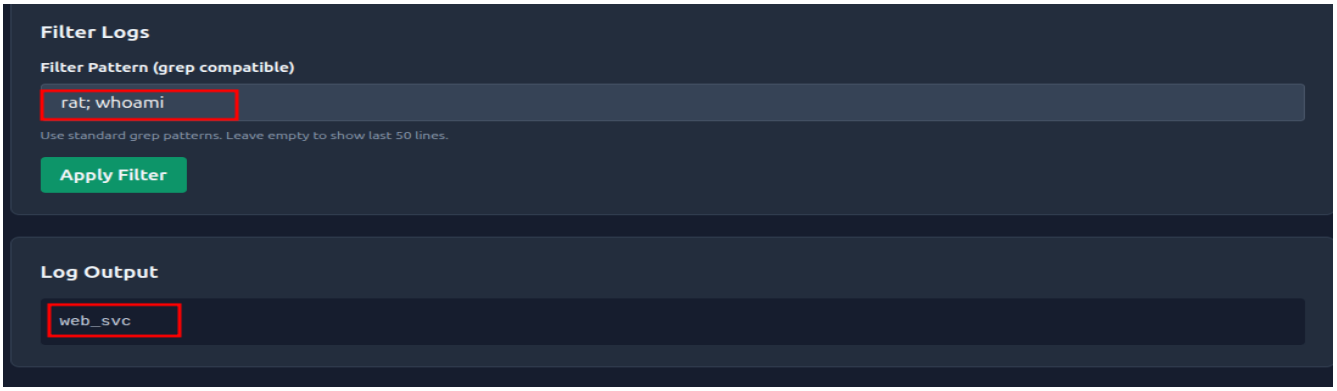
Security Impact:
Complete server-level compromise. Arbitrary command execution as the web_svc user. Full filesystem read access including application source (app.py), database (smallmart.db), user flag (user_flag.txt), and root flag (ROOTFLAG). A rootbash binary was also present on the server.

Business Impact:
Full server compromise represents the most severe business risk scenario, potentially resulting in total loss of system integrity, unauthorized access to internal networks, theft of proprietary source code and intellectual property, and the ability to deface the website or deploy ransomware.

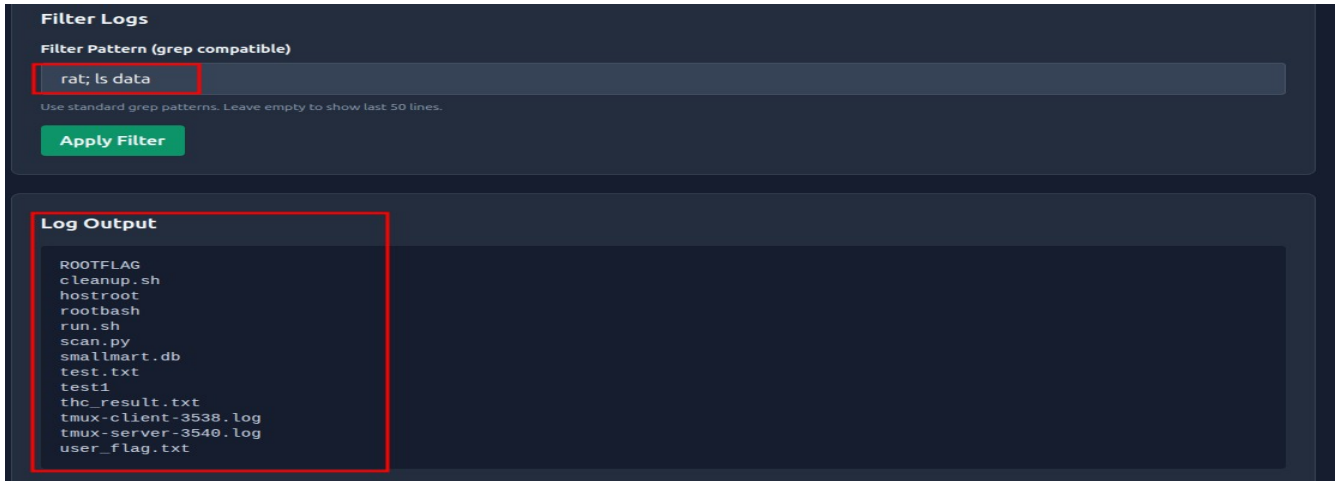
Remediation Recommendation:
- The preferred remediation is to remove web-based operating -system command execution entirely if not required.
- Never pass user input to shell commands. Replace shell grep with pure python implementation
- Apply principle of least privilege and restrict web server file access.

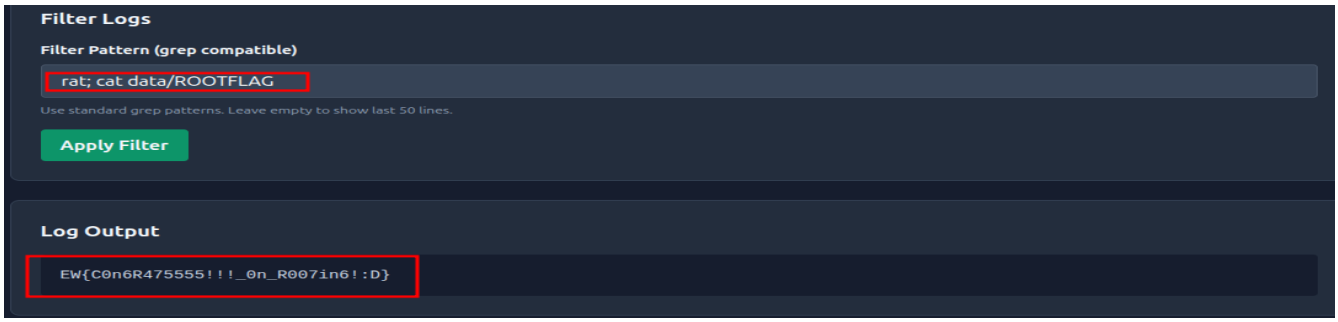
Steps to Reproduce:
1. Obtain the admin credential via SQL injection by dumping the users database table.
2. Authenticate to admin panel using recovered credential.
3. Enter command injection payload: rat; id; whoami
4. Click apply filter, observe OS command output in Log Output section.
5. Execute: rat; ls -la data/ to enumerate server file system.
6. Execute: rat; cat data/user_flag.txt to read user flag.
7. Execute: rat; cat data/ROOTFLAG to read root flag.

Supporting Evidence:









F-02: SQL Injection in product ID endpoints leading to database dump

Insecure Password Storage in unsalted MD5 hashing	
CVSS Score	7.1 (High)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:L/VA:L/SC:N/SI:N/SA:N
Affected Component	Product UUID Functionality
Affected Host	https://shop.rezydev.com/products?product=94c3aff1-c517-4394-ade6-23399aeab78e%27OR%201=1--

Description:
The product detail endpoint passes the user-supplied product UUID parameter directly into a SQLite query without parameterization. This unauthenticated SQL injection allows boolean-based blind and time-based blind extraction of the complete database. The users, products, flags and sqlite_sequence tables were enumerated during dumpling the database tables with SQLMap. The users table contains administrative credentials including admin account with MD5 hashed passwords.

Security Impact:
This vulnerability allows an unauthenticated external attacker to dump the complete application database through boolean-based blind SQL injection. The attacker can retrieve all user credentials, administrative passwords, customer data and sensitive application flags without any authentication or user interaction.

Business Impact:
Complete exposure of customer personally identifiable information (PII) and business-sensitive data could result in severe regulatory non-compliance with data protection requirements, significant financial penalties, permanent reputational damage and loss of customer trust.

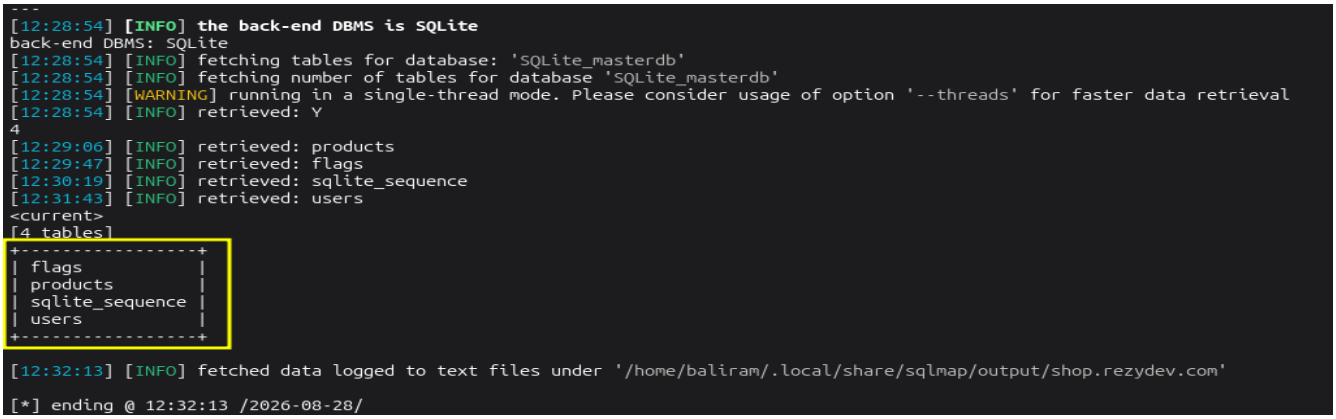
Remediation Recommendation:
- Implement parameterized queries for all database operations immediately
- Validate and sanitize all user-supplied input on the server side. Reject the dangerous character like ‘, ”, ;, --, ‘)).
- Conduct full database audit. Deploy WAF as supplementary defence.
- Never concatenate user-controlled input directly into SQL statements.

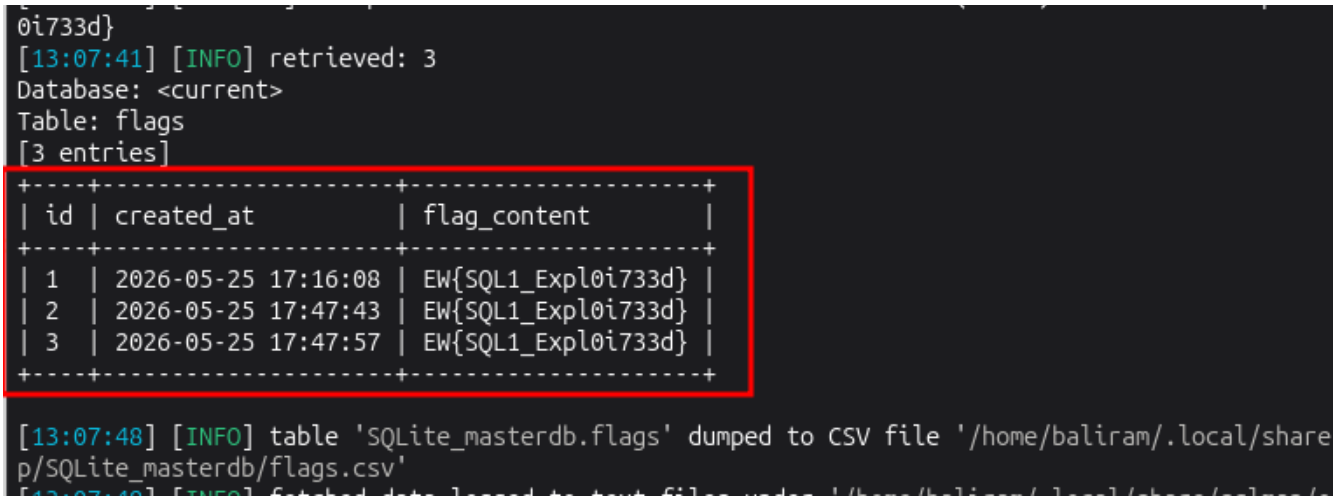
Steps To Reproduce:

1. Identify the product details endpoint: /proudcts?product={uuid}
2. Append the single quote to UUID, it returns with error that confirms sql injection.
3. Capture HTTP request in Burp Suite and save as .txt file.
4. Run: sqlmap -r ctf-request.txt --tables --batch dbms.
5. Confirm boolean based blind and time-based blind injection.
6. Dump users table to retrieve the users details.
7. Dump flags table to retrieve application flags.

Supporting Evidence:







F-03: Insecure Password Storage in unsalted MD5 hashing

SQL Injection in product ID endpoints leading to database dump	
CVSS Score	7.1 (High)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:L/VA:L/SC:N/SI:N/SA:N
Affected Component	Product UUID Functionality
Affected Host	https://shop.rezydev.com/products?product=94c3aff1-c517-4394-ade6-23399aeab78e%27OR%201=1--

Description:
Users password in the SQLite database are stored as unsalted MD5 hashes. MD5 is a cryptography broken algorithm entirely unsuitable for password storage. Unsalted MD5 hashes are instantly reversed using precomputed rainbow tables and public cracking services such as CrackStation, requiring no computational effort.

Security Impact:
Users password stored as unsalted MD5 hashes can be instantly reversed using precomputed rainbow tables or public cracking tools. This cryptographic failures provides zero protection against offline dictionary attacks, allowing attacker who gains database access to recover all user passwords in seconds including administrative credentials.

Business Impact:
Credential compromise enables attacker to perform credential stuffing attacks against external services, potentially impacting users who reuse password across multiple platforms. The organization faces regulatory non-compliance with NIST SP 800-63B and PCI-DSS requirements exposing the business fines, lawsuits and mandatory breach notification.

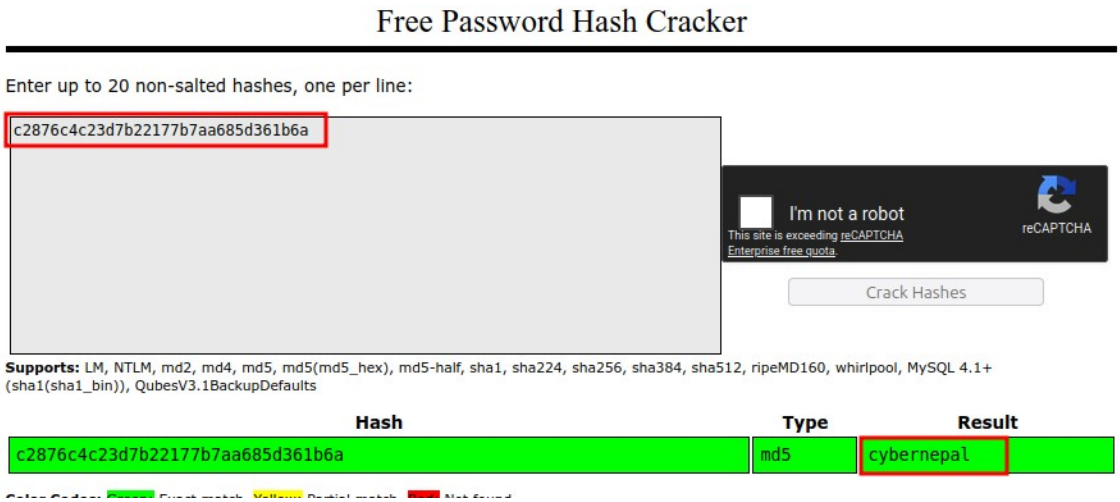
Steps To Reproduce:
1. Dump the password hashes via SQL Injection.
2. Identify the hash format: 32 hexadecimal characters MD5
3. Submit hashes to https://crackstation.net
4. Admin and user passwords recovered in under 1 second, no computation required.

Remediation Recommendation:

- Replaces MD5 with Argon2id, bcrypt or scrypt. Enforce a minimum cost factor.
- Force a password reset for all users and re-hash on next login.

Supporting Evidence:

```
[06:43:59] [INFO] fetching number of column(s) 'password,username' entries for table 'users' in database 'SQLite_masterdb'
[06:43:59] [WARNING] running in a single-thread mode. Please consider usage of option '--threads' for faster data retrieval
[06:43:59] [INFO] retrieved: 107
[06:44:14] [INFO] retrieved: c2876c4c23d7b22177b7aa685d361b6a
[06:47:14] [INFO] retrieved: ' OR '1'='1'--+-
[06:48:45] [INFO] retrieved: 827ccb0eea8a706c4c34a16891f84e7b
[06:51:44] [INFO] retrieved: ' OR 1=1--+-
[06:52:53] [INFO] retrieved: 25ce^Ce^d1ee12c344f4404aba5b703e25
[06:55:50] [INFO] retrieved: 01532539
[06:56:38] [INFO] retrieved: 827ccb0eea8a706c4c34a16891f84e7b
[06:59:44] [INFO] retrieved: 1d3d2d231d2dd4
[07:00:56] [INFO] retrieved: 827ccb0eea8a706c4c34a16891f84e7b
[07:04:01] [INFO] retrieved: Admin
[07:04:29] [INFO] retrieved: 7a5e5453e31147a10f87ed514929eaaf
[07:07:36] [INFO] retrieved: Goku
[07:07:58] [INFO] retrieved: 6e809cbda0732ac4845916a59016f954
[07:11:01] [INFO] retrieved: Hajurbakonati11
[07:12:17] [INFO] retrieved: 6ca29d9bb530402bd09fe026ee838148
[07:15:20] [INFO] retrieved: Hero
[07:15:43] [INFO] retrieved: 04227103be7ae0fc17b5721be76593be
[07:18:43] [INFO] retrieved: OPP
[07:19:01] [INFO] retrieved: 82b149009b9c75b9d21b9a593fefe6d8
[07:22:01] [INFO] retrieved: Poo
[07:22:20] [INFO] retrieved: 6fc42c4388ed6f0c5a91257f096fef3c
[07:25:33] [INFO] retrieved: Poonam
[07:26:05] [INFO] retrieved: 6fc42c4388ed6f0c5a91257f096fef3c
[07:29:14] [INFO] retrieved: Spandan
[07:29:52] [INFO] retrieved: 6fc42c4388ed6f0c5a91257f096fef3c
[07:33:01] [INFO] retrieved: Test_User
[07:33:50] [INFO] retrieved: 6fc42c4388ed6f0c5a91257f096fef3c
[07:36:53] [INFO] retrieved:
```



API endpoints exposed in JavaScript	
CVSS Score	5.3 (Medium)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:L/VA:L/ SC:N/SI:N/SA:N
Affected Component	Static JS Files
Affected Host	https://shop.rezydev.com/static/index.js

Description:
The publicly accessible file /static/index.js exposes internal API endpoint paths and application logic. The script reveals the backend API structure used by the frontend, including the product listing and detail endpoints, enabling direct API interaction bypassing the UI layer.

Security Impact:
The publicly accessible JavaScript file exposes internal API endpoint paths and application logic, enabling direct API interaction bypassing client-side validation and rate limiting. Attackers can leverage this information to perform automated attacks and enumerate all application resources without UI restrictions.

Business Impact:
Direct API access enables data scraping, automated exploitation of injection vulnerabilities, and bypassing of security controls implemented only at the UI layer. This exposes the business to automated attacks and potential denial-of-service through unauthenticated API abuse.

Steps To Reproduce:
1. Identify the homepage and identify the linked JavaScript files.
2. Fetch /static/index.js by using curl -sk https://shop.rezydev.com/static/index.js
3. Analysis a source code, API endpoints products logic are fully exposed.
4. Flags discovered in JS comment confirming sensitive content in client-side code.

Remediation Recommendations:
- Avoid hardcoding API endpoints or sensitive logic in client-side JavaScript.
- Implement server-side authentication in all API endpoints.
- Use bundling/minification with dead code elimination in production builds.

Supporting Evidence:

```
:~$ curl -sk https://shop.rezydev.com/static/index.js
// EW{J5_F1l3s_Ar33_Mu57}

// Load products from API and display them
async function loadProducts() {
    try {
        const response = await fetch('/api/products');
        const products = await response.json();

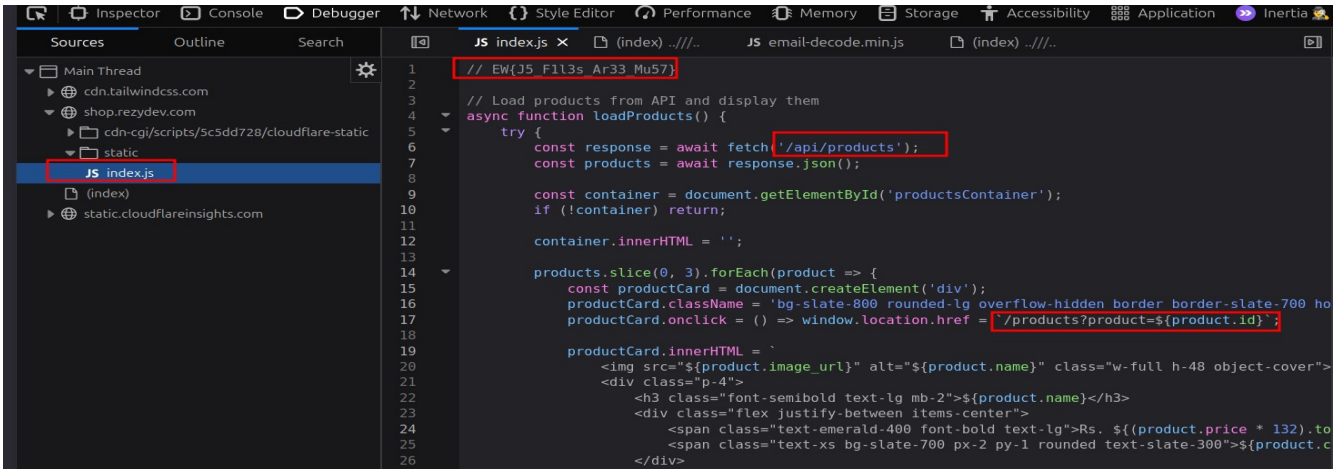
        const container = document.getElementById('productsContainer');
        if (!container) return;

        container.innerHTML = '';

        products.slice(0, 3).forEach(product => {
            const productCard = document.createElement('div');
            productCard.className = 'bg-slate-800 rounded-lg overflow-hidden border border-slate-700 hover:border-emerald-600 tra
nsition hover:shadow-lg hover:shadow-emerald-500/20 cursor-pointer';
            productCard.onclick = () => window.location.href = `/products?product=${product.id}`;

            productCard.innerHTML = `
                <img src="${product.image_url}" alt="${product.name}" class="w-full h-48 object-cover">
                <div class="p-4">
                    <h3 class="font-semibold text-lg mb-2">${product.name}</h3>
                    <div class="flex justify-between items-center">
                        <span class="text-emerald-400 font-bold text-lg">Rs. ${(product.price * 132).toFixed(2)}</span>
                        <span class="text-xs bg-slate-700 px-2 py-1 rounded text-slate-300">${product.category}</span>
                    </div>
                </div>
            `;

            container.appendChild(productCard);
        });
```



F-05: Sensitive Data exposure in source code comment left by developer

Sensitive Data exposure in source code comment left by developer	
CVSS Score	5.3 (Medium)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:L/VA:L/ SC:N/SI:N/SA:N
Affected Component	Source Code
Affected Host	https://shop.rezydev.com/

Description:
During reconnaissance and source code analysis, developer provided information was discovered within application source materials. The application's HTML source code contains an HTML comment embedding a sensitive value in the footer of every page. This comment's is rendered in the DOM and visible to any unauthenticated user who views the page source via browser developer tools or curl.

Security Impact:
Developer comments can expose internal logic, hardcoded credentials, API endpoints or sensitive configuration. This finding demonstrates a lack of pre-deployment source code review.

Business Impact:
Internal developer notes visible to external parties may reveal application architecture or sensitive business logic to malicious actors. Attacker can use this information for social engineering

Steps To Reproduce:

1. Navigate to https://shop.rezydev.com/ in a browser.
2. View page source (Ctrl+U) or use curl to fetch the raw HTML.
3. Search for HTML comments.
4. Sensitive developer comment found in page footer on every page.

Remediation Recommendation:

- Implement pre-deployment code review to strip all HTML comments and debug output.
- Integrate automated secret scanning (Semgrep, git-secrets, truffleHog) into the CI/CD pipeline.

Supporting Evidence:



Sensitive Data in HTTP Response Header

Sensitive Data in HTTP Response Header	
CVSS Score	5.3 (Medium)
CVSS Vector	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:L/VA:L/SC:N/SI:N/SA:N
Affected Component	HTTP Response Header
Affected Host	https://shop.rezydev.com/login

Description:
Upon the successful authentication, the application returns a custom HTTP response header X-Flag for containing a sensitive flag. The Flask session cookie also stores role information (is_admin) in a client-readable base64-encoded format. Analysis in Burp Suite confirmed the flag is delivered in the response header rather than the response body.

Security Impact:
Custom response headers containing sensitive values may be captured by network intermediaries, proxies or browser extensions. Client-readable role claims in cookies may facilitate privilege escalation if the signing secret is weak.

Business Impact:
Leakage of authentication metadata assists attackers in understanding the access control model and preparing subsequent escalation attack.

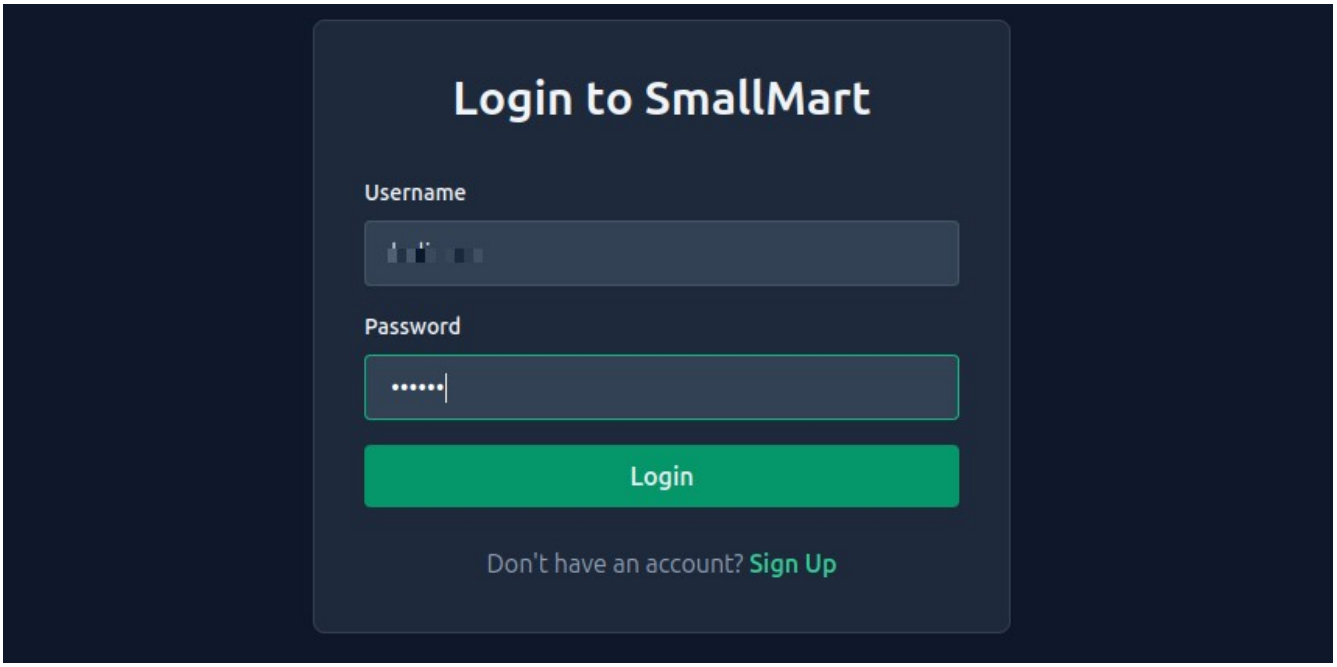
Steps To Reproduce:

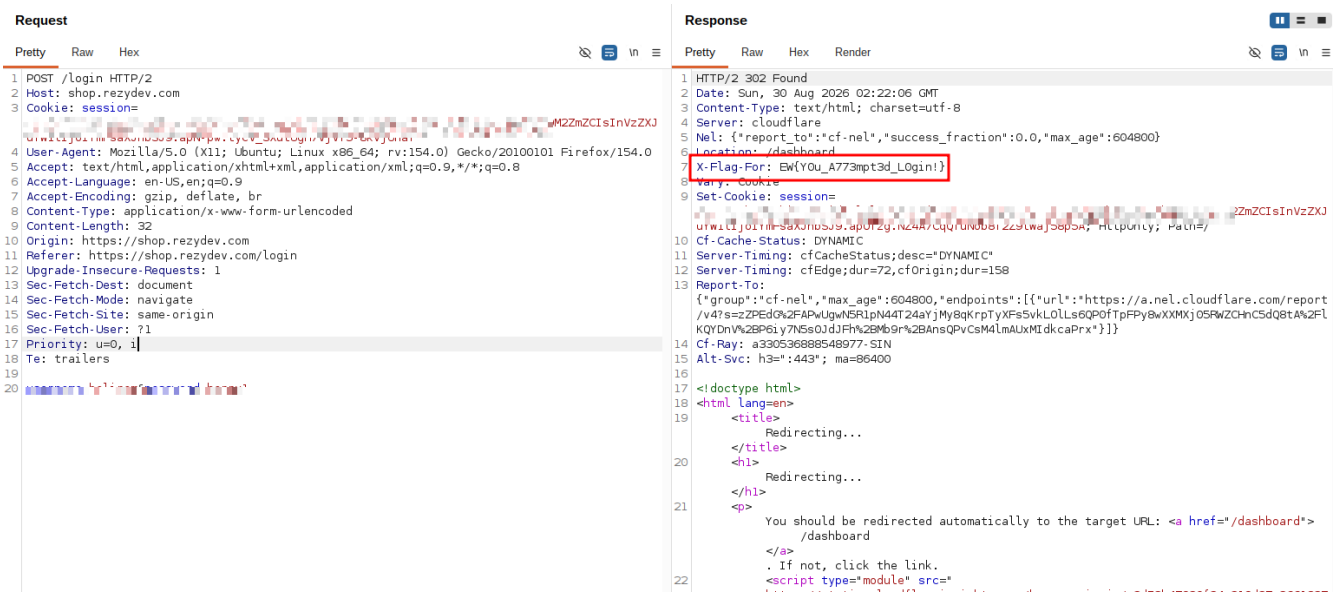
1. Access the application in browser.
2. Login to the application.
3. Start a Burp suite and capture the login request.
4. Analyze the HTTP response.

Remediation Recommendation:

- Remove all sensitive data from HTTP response headers. Use server-side sessions (Redis-backed).
- Ensure Flask SECRET_KEY is cryptographically strong and rotated regularly.

Supporting Evidence:





7. Conclusion

The penetration testing assessment of the SmallMart web application successfully identified and validated multiple security weaknesses that could be chained to gain unauthorized access to sensitive application functionality. The assessment covered reconnaissance, enumeration, source-code analysis, authentication-flow analysis, SQL injection testing, privilege escalation and post-exploitation activities.

The most significant vulnerability identified was SQL Injection, which enabled interaction with the application's SQLite database and enumeration of sensitive tables such as users and flags. Further analysis of the application's authentication and authorization mechanisms allowed access to privileged functionality including the Secret Panel and administrative /view-logs interface. Post-exploitation activities provided additional access to sensitive server-side resources and ultimately enabled completion of the CTF objectives, including retrieval of the new root.txt flag.

The assessment demonstrates that weaknesses in input validation, authentication, authorization, and administrative functionality can significantly increase the risk of application compromise when exploited in combination. The primary remediation priorities should therefore be to implement parameterized SQL queries, strengthen server-side authorization, secure administrative endpoints, restrict command execution functionality, protect sensitive files, and enforce least-privilege principles.

Overall, the assessment highlights the importance of defense-in-depth and secure application development practices. Addressing the identified vulnerabilities and conducting a follow-up security assessment would significantly improve the application's resistance to similar attack techniques.